

Implementation Handbook — Volume 1

Sealed Enterprise Architecture

Measurement · Enforcement · Verification · Evolution

Operational guidance for organisations adopting SEA 1.0

Volume 1 · April 2026 · Suneesh Sidharthan

Companion to SEA 1.0

This handbook assumes familiarity with the SEA 1.0 whitepaper. It does not restate the philosophy — it addresses the operational questions that arise naturally once an organisation has decided to adopt SEA.

Four questions drive this volume: How do we measure whether SEA is working? How do we enforce sealed contracts in practice? How do we verify implementations conform to contracts? How do we evolve sealed systems without losing their sealed character?

This is Volume 1. A planned Volume 2 will address team-scale operations, SDLC integration patterns, and reference templates once early adoption experience has informed them.

Contents

This handbook is organised in four parts, each addressing one operational dimension of adopting SEA.

§	Section	Description
	FRONT MATTER	
	Preface	What this handbook is, and is not
	How to Read This Handbook	Different readers need different paths through it
	PART I — MEASUREMENT	
1	Why SEA Needs Its Own Metrics	Standard software metrics do not capture what SEA protects
2	The Contract Integrity Score	A composite metric for a single sealed contract
3	Portfolio-Level Health Indicators	Metrics that apply across all sealed contracts
4	Governance Health Indicators	Metrics for the protocols rather than the artefacts
5	Leading vs Lagging Indicators	Predicting drift vs confirming it
6	Anti-Metrics	Numbers that look like health but measure activity
	PART II — ENFORCEMENT	
7	What Enforcement Means in Practice	The distinction between protocol and physical prevention
8	Repository-Level Enforcement	Git, branch protection, CODEOWNERS, signing
9	Build-Pipeline Enforcement	CI/CD integration, hash verification, merge gates
10	Registry Patterns	Where sealed contracts physically live
11	Breach Response	What to do when enforcement fails
	PART III — VERIFICATION	
12	The Verification Layer, Expanded	Beyond PASS/FAIL
13	Five Compliance States	A richer vocabulary than binary conformance
14	Severity Classification for Violations	Four levels, with operational meaning
15	Static vs Runtime Verification	Different mechanisms for different guarantees
16	Cross-Contract Verification	Dependencies and the relationships between contracts
	PART IV — EVOLUTION	
17	The Evolution Layer in Depth	What the whitepaper left abstract

§	Section	Description
18	Extension Pattern Catalogue	The common shapes extensions take
19	The Parallel-Live Playbook	Operating two contracts simultaneously
20	Rollback Patterns	When a new contract proves flawed mid-transition
21	Major Version Bump — Detailed Protocol	Stage-by-stage operational guidance
22	Retirement Procedures	How sealed contracts end with dignity

Preface

The SEA 1.0 whitepaper articulated a philosophy. It named the gaps in current architectural practice, specified a five-layer framework, and described the governance protocols that hold it together. It was deliberately written as a foundational document — the kind that gets quoted, cited, and argued with over years.

This handbook is the other kind of document. It is operational. It assumes the reader has accepted the philosophy and now needs guidance on the mechanics: what does measurement actually look like, what does enforcement physically mean, how does verification work beyond the abstract, how does evolution happen safely. The questions this handbook addresses are not philosophical. They are the questions a platform team asks during the first six months of adoption.

I want to be direct about the status of what follows.

The operational guidance in this handbook is first-draft design thinking, applied rigorously, but not battle-tested at enterprise scale. The metrics I propose are reasonable but unvalidated against real portfolio data. The enforcement patterns will work in theory; real organisations will find edge cases the theory did not anticipate. The verification layer specification is complete in structure but the actual per-stack implementations will need real engineering work to validate. The evolution patterns are derived from first principles and from one solo validation case; multi-team adoption will surface situations I have not modelled.

This is the honest limit of what documentation can do without real deployment. I could pretend otherwise, produce confident-sounding guidance, and trust that readers will not notice the gap between design and validation. I will not. The handbook should do what it can do — give organisations a coherent starting point for operational decisions — without claiming to replace the experience of actually adopting SEA.

Theory grounds practice. Practice tests theory. Neither can substitute for the other.

A second volume is planned. It will address team-scale operations under SEA, explicit integration with the AI-Native SDLC, the reference templates organisations need for day-to-day operations, and worked examples across sectors. That volume will be written after early adoption experience has revealed what this one missed. Producing it now would be speculation; producing it later will be guidance informed by what actually happens.

Readers who want rapid adoption should begin with Parts I and II of this handbook — measurement and enforcement address the most urgent operational questions. Parts III and IV can follow as the adoption matures. The order within each part is sequential, but the parts themselves can be approached in whatever order serves the organisation's current situation.

One note on tone: the handbook mixes practitioner guidance (do this, check that, watch for this pattern) with the same reflective register the whitepaper used for philosophical arguments. This is deliberate. Operational decisions rest on underlying principles, and naming those principles inside the operational guidance — rather than assuming the reader will flip back to the whitepaper — keeps the handbook self-contained for the specific operational question at hand.

How to Read This Handbook

Different readers come to a handbook with different needs. Four common paths through this one:

If you are a CTO considering adoption

Read the Preface, then skip to Part I (Measurement). Metrics are how you will justify the adoption decision to your executive team and measure whether adoption is working. Then skim Part II (Enforcement) to understand the infrastructure implications. Parts III and IV matter for your platform team, not for the decision you are making now.

If you are a platform team lead preparing to implement

Read in order: Preface, then Parts II, III, IV, then I. Enforcement and verification are the infrastructure you will build or configure. Evolution is the process you will facilitate. Measurement is how you will demonstrate results to leadership once the first three are in place.

If you are a sealed contract owner

Read the Preface, then Part I (because you will be measured) and Part IV (because your contracts will evolve). Parts II and III describe infrastructure that exists around you but is not directly your concern.

If you are an enterprise architect integrating SEA with existing practice

Read the Preface, then Parts III and IV in depth — the verification and evolution layers are where SEA intersects with your existing architectural governance. Parts I and II are supporting context.

A note on what is not in this volume

This handbook does not cover: multi-team operations and governance disputes, explicit SDLC integration patterns, the full set of reference templates (contract YAML template, intent document template, extension proposal template, quarterly review agenda, and so on), worked sector examples, or the anti-pattern catalogue for common adoption mistakes.

Those topics are scoped into Volume 2, which will be written after early adopters have surfaced the real questions in those areas. Pre-emptive answers without adoption experience would be fiction dressed as guidance.

PART I

Measurement

How do we know SEA is working? How do we demonstrate it to leadership? How do we catch drift before it becomes failure?

1. Why SEA Needs Its Own Metrics

Most software engineering metrics were designed for a world where code was the primary artefact, delivery velocity was the primary concern, and human engineers were the primary authors. Story points, lead time, change failure rate, mean time to recovery — these metrics assumed stable contracts, stable teams, stable purposes. They measured the transient layer of the system without any awareness that a durable layer existed.

SEA shifts the centre of architectural concern from the transient to the durable. Standard metrics do not capture whether the durable layer is healthy, because they were never designed to. An organisation can have excellent delivery metrics — low lead time, high deployment frequency, low change failure rate — while its sealed contracts silently decay, its intent documents go unmaintained, and its governance protocols become theatre. The delivery metrics will look healthy until the moment the architectural debt makes further delivery impossible.

This is not a hypothetical pattern. It is the standard story of enterprise systems that appear healthy for years and then, in a single quarter, become impossible to maintain. The delivery metrics did not warn anyone because the delivery metrics were measuring the wrong layer.

A system with excellent delivery metrics and rotting sealed contracts is indistinguishable from a healthy system, until it is not.

1.1 What Good SEA Metrics Must Do

SEA-specific metrics must satisfy four properties:

1. They must measure the durable layer. Metrics that only count activity in the transient layer — pull requests, deployments, story points — cannot tell you whether the sealed contracts are holding.
2. They must detect drift early. The value of architectural measurement is not in confirming that a collapse has occurred but in detecting the conditions that will produce collapse before it happens.
3. They must resist gaming. Any metric an organisation adopts will be optimised for. The chosen metrics must be difficult to improve through activity that does not actually improve architectural health.
4. They must be interpretable by non-specialists. Executives, product managers, and engineers outside the platform team must be able to read the metrics without requiring architectural training. Metrics that only specialists can interpret will not be trusted, and untrusted metrics will not drive decisions.

1.2 Three Layers of Measurement

This handbook proposes SEA measurement at three distinct layers, each addressing a different question:

Layer	Question	Measured units
Contract-level	Is this specific sealed contract healthy?	One score per contract (the Contract Integrity Score, Section 2)
Portfolio-level	Is the full set of sealed contracts	Distribution statistics across all contracts

Layer	Question	Measured units
	healthy?	(Section 3)
Governance-level	Are the protocols themselves working?	Activity and timing metrics for review processes (Section 4)

The three layers are not redundant. A portfolio can be healthy overall while individual contracts rot. Governance can be active while contract health declines. Each layer catches patterns the others miss. An organisation adopting SEA should operate all three, with different stakeholders responsible for different layers — contract owners for their own contracts, the platform team for the portfolio, the Sealed Review Board for governance.

2. The Contract Integrity Score

The Contract Integrity Score is a composite number between 0 and 100 that summarises the health of a single sealed contract. It is calculated from six sub-metrics, each weighted according to how much that aspect matters to overall integrity. A contract at 100 is fully healthy across all dimensions. A contract below 70 needs attention. A contract below 50 is at risk of failure and should be a priority for the next Sealed Review Board meeting.

The score is designed to be readable at a glance, to resist gaming, and to surface the specific sub-dimensions in which a contract is weakening so that intervention can be targeted.

2.1 The Six Sub-Metrics

Each sub-metric is scored 0 to 100 independently. The Contract Integrity Score is their weighted average.

Sub-metric	What it measures	Weight
Intent Completeness	How completely the intent document covers the contract's sections (problem, invariants with rationale, rejected alternatives, assumptions)	25%
Change History Cleanliness	Whether the change history is current, sequential, properly attributed, and free of ungoverned modifications	20%
Invariant Coverage	For each declared invariant, whether there is a test, assertion, or runtime check that would detect violation	20%
Ownership Currency	Whether the named Contract Owner and Reviewer are currently active in the role and have been confirmed in the last review cycle	15%
Consumer Alignment	Whether known consumers of the contract are using the current version, or are stale on older versions	10%
Review Cadence Adherence	Whether the contract has been reviewed in the last quarterly cycle	10%

2.2 Calculating Each Sub-Metric

Intent Completeness (weight 25%)

The intent document is the single most valuable artefact in SEA. An inscrutable contract with a missing intent document is worse than no contract at all — it is a rule that cannot be questioned because its reasoning is lost. Weighting intent most heavily reflects this.

Intent Completeness is scored by checking presence and minimum length of the required sections:

- Problem Statement present and at least 100 words: 20 points
- Invariants listed with rationale for each: 25 points (pro-rated if some invariants lack rationale)
- Rejected Alternatives section with at least two alternatives: 20 points
- Assumptions section with at least three assumptions: 15 points

- Failure Modes Protected Against: 15 points
- Related Context with links to decision records or incidents: 5 points

The thresholds are deliberately modest. A contract whose intent document contains even first-draft content in each section scores around 70. A fully developed intent document scores near 100. A contract with no intent document scores zero — which, by design, dominates the overall Contract Integrity Score and surfaces the problem immediately.

Change History Cleanliness (weight 20%)

The change history in a sealed contract YAML is append-only. Its cleanliness reflects whether governance has been exercised rigorously over the contract's lifetime.

- Every entry has a date, action, actor, and notes: 30 points
- No gaps in the sequence (no period longer than 6 months without an entry if the contract has active consumers): 20 points
- Actions match the allowed enum (sealed, extended, marked-parallel-live, deprecated, retired): 15 points
- No evidence of ungoverned modification (no diff between the file's state and what the change history declares): 20 points
- Major events (seals, major version bumps) reference the authorising body: 15 points

Invariant Coverage (weight 20%)

A declared invariant without an enforcement mechanism is a wish, not a guarantee. Invariant Coverage measures how many declared invariants have corresponding verification mechanisms.

- For each invariant, identify whether it has: (a) a unit or integration test, (b) a runtime assertion, (c) a build-time static check, or (d) a manual review checkpoint documented in the governance protocol
- Score: (invariants with at least one mechanism / total invariants) × 100

This sub-metric requires modest manual review to calculate accurately. Automating it fully is possible only for contracts whose invariants are formally expressed in machine-readable form. For most contracts, a platform team member reviews the invariants against the test suite quarterly and records the score.

Ownership Currency (weight 15%)

A contract whose named Owner has left the organisation is an orphaned contract, even if the name is still in the YAML. Ownership Currency measures whether the governance of the contract is actually active.

- Current Owner is a current employee or equivalent in the organisation: 40 points
- Current Owner has participated in the last quarterly review: 20 points
- Current Reviewer is a current employee and different from Owner: 25 points
- Both Owner and Reviewer have been confirmed in their roles within the last annual cycle: 15 points

Consumer Alignment (weight 10%)

A sealed contract's purpose is to serve consumers. If all consumers have migrated off, the contract is a candidate for retirement. If consumers are scattered across multiple versions, the contract is in a parallel-live state that may have become permanent.

- Known consumer count is greater than zero: 30 points (if zero, consider retirement)

- All known consumers are on the current contract version: 40 points
- Any deprecated consumer implementations have documented migration plans: 30 points

Review Cadence Adherence (weight 10%)

SEA's governance protocol specifies quarterly review. Cadence Adherence measures whether this is actually happening for the specific contract.

- Contract was reviewed in the last quarterly Sealed Review Board cycle: 60 points
- Any issues identified in the last review have been addressed or have an open remediation ticket: 40 points

2.3 Interpreting the Composite Score

The weighted average produces a single number that can be interpreted as follows:

Score range	Interpretation	Action
90–100	Healthy	No action needed. Continue normal governance cycle.
75–89	Generally healthy, specific sub-metrics may need attention	Identify the lowest sub-metric and address it in the next review cycle.
60–74	Weakening. Some aspect of contract integrity is declining.	Contract Owner addresses the weak sub-metrics within 30 days. Report to SRB.
40–59	At risk. The contract is losing its sealed character.	SRB-level attention. Review whether the contract needs renewed governance, re-ownership, or candidacy for retirement.
0–39	Failing. The contract exists on paper but not in practice.	Immediate SRB review. Either restore the contract to health or retire it formally. Do not leave it in this state.

2.4 A Worked Example

To make the scoring concrete, consider a hypothetical contract ``auth.user_session.v1`` after one year of operation.

Contract: `auth.user_session.v1`
Reviewed: 2026-04-22

Intent Completeness:	82 / 100	(weight 25%)	20.5
- Problem Statement: present, 180 words	20 / 20		
- Invariants with rationale: 3/3 with rationale	25 / 25		
- Rejected Alternatives: 2 alternatives	20 / 20		
- Assumptions: 2 only (below threshold of 3)	10 / 15		
- Failure Modes Protected: missing	0 / 15		
- Related Context: present	5 / 5		

Change History Cleanliness: 95 / 100 (weight 20%) 19.0

- Complete metadata on all entries	30 / 30	
- No gaps	20 / 20	
- Allowed actions only	15 / 15	
- One recent diff without change entry – flagged	15 / 20	
- Major events reference authority	15 / 15	
Invariant Coverage:	67 / 100 (weight 20%)	13.4
- 2 of 3 invariants have enforcement mechanisms		
Ownership Currency:	100 / 100 (weight 15%)	15.0
Consumer Alignment:	70 / 100 (weight 10%)	7.0
- 5 consumers, 4 on current version, 1 on v0		
- No migration plan documented for the v0 consumer		
Review Cadence:	100 / 100 (weight 10%)	10.0
Contract Integrity Score:		84.9
Interpretation: Generally healthy. Two specific actions:		
1. Extend intent document with Failure Modes section		
2. Add enforcement for the third invariant		
3. Document migration plan for the v0 consumer		

2.5 How the Score Is Used

The Contract Integrity Score is a conversation starter, not a verdict. Three practical uses:

5. Dashboard display. Every sealed contract's current score, updated at least quarterly, visible to the Sealed Review Board and to engineering leadership.
6. Quarterly review input. When the SRB meets, contracts below 75 are the first to be discussed. Contracts below 60 require explicit remediation plans before the meeting closes.
7. Trend tracking. A contract's score over time matters more than its absolute value. A contract at 85 trending downward over three quarters is a more urgent concern than a contract at 75 that has been stable for two years.

Do not let the score replace judgement

The Contract Integrity Score is a useful summary. It is not a substitute for the Sealed Review Board reading the actual contract and its intent document, speaking with the Owner, and forming a qualitative view of the contract's health.

A contract can score 90 and still be drifting from its original purpose in ways the sub-metrics do not capture. Another contract can score 70 for defensible reasons — perhaps the organisation has deliberately deferred certain intent sections pending a known future decision.

Use the score to focus attention. Use judgement to decide what the attention reveals.

3. Portfolio-Level Health Indicators

The Contract Integrity Score tells you whether a single contract is healthy. Portfolio-level indicators tell you whether the full set of sealed contracts is healthy as a system. These are different questions. A portfolio can contain mostly-healthy contracts while still showing systemic problems — for instance, if all contracts are owned by the same person, or if the average contract age is declining suggesting premature retirement.

Seven portfolio-level indicators matter most. Each answers a distinct question about the system as a whole.

3.1 Contract Age Distribution

Plot the age of each sealed contract from its sealing date to the present. The distribution tells a story.

A healthy portfolio shows a distribution that roughly matches the organisation's expected contract lifetime, with a slight bias toward older contracts (the oldest ones being the most important ones, since they have survived the longest). An unhealthy portfolio shows one of several dysfunctional patterns:

- All contracts are recent (under 2 years old) — the organisation may be over-sealing, sealing contracts that should have been left transient, or experiencing high contract churn that defeats the purpose of sealing
- Contracts cluster in a single cohort (all sealed within a few months) — typical of an adoption burst followed by stagnation; new architectural work is not being sealed
- No contracts are recent — new architectural work is happening but not being sealed, suggesting the practice has become theatre for the existing portfolio while new work bypasses it

3.2 Drift Incidents per Quarter

A drift incident is a documented case of divergence: a contract and its implementation no longer match, two artefacts that should agree do not, or a sealed contract has been modified outside the governance protocol. Every drift incident is numbered (CD-001, CD-002, and so on) and investigated.

The per-quarter count is the single most important portfolio-level indicator. A healthy organisation will have a small number of drift incidents each quarter — not zero, because zero usually means the organisation is not looking carefully. A trend upward is a warning. A trend downward is progress.

Sub-categorisation matters. Incidents should be classified by root cause: verification gap (the verifier did not catch it), governance gap (the review process allowed it), education gap (the author did not know the rule), or tooling gap (the enforcement mechanism failed). The category distribution tells the organisation where to invest.

3.3 Extension Approval Rate

Of all extension proposals submitted, what fraction are approved? This metric is most useful when examined alongside the approval time.

A very high approval rate (above 95%) combined with fast approval suggests the extension review is not substantive. A very low rate (below 60%) suggests either the review is too strict or the proposals are poorly prepared — both of which slow legitimate work. The healthy range is approximately 75–90%, with an average approval time of 3–7 working days.

An organisation should track the distribution of rejection reasons. Common rejection reasons are:

- Extension falls outside the declared extension policy — the Owner rejects and either the proposal is revised or the Extension Review Board considers whether the policy should be revised
- Invariants would be weakened — the Reviewer rejects; this is the strongest kind of rejection and must not be overridden without major review
- Proposal is under-specified — returned for rework, not rejected in substance
- Extension conflicts with another in flight — coordination issue, not rejection in principle

3.4 Succession Event Metrics

Every ownership change is a succession event. The metrics are: number of events per year, average duration of the shadow period, and post-succession integrity (has the new Owner maintained or improved the contract's Integrity Score in the first quarter of ownership).

Succession is where most institutional-memory loss happens. An organisation that performs succession well will show steady ownership transitions that do not produce visible score degradation. An organisation that performs it poorly will show a pattern where every succession is followed by a drop in the Contract Integrity Score of the affected contracts, recovering slowly or not at all.

3.5 Parallel-Live Duration Distribution

When a contract is in parallel-live state (an old version and a new version both operational during migration), how long does it typically stay there? A healthy distribution shows most parallel-live periods resolving within 6–12 months, with a long tail for complex migrations.

Parallel-live states that persist for years indicate a failed migration — the new contract has not actually replaced the old, the organisation has simply accumulated both. This is one of the failure modes the whitepaper named as Decommissioning Paralysis. Tracking parallel-live duration catches it before it becomes a permanent feature of the architecture.

3.6 Orphan Contract Count

An orphan contract is one with no active Owner, no active consumers, or both. Orphans accumulate silently — people leave, consumers migrate away, nobody formally retires the contract. A portfolio should have zero orphans, ideally, or else have a documented process that converts any discovered orphan into either a re-owned contract or a retired contract within one quarterly cycle.

3.7 Major Version Bump Frequency

Major Version Bumps (MVBs) should be rare — approximately one per five to ten years for an enterprise system. An organisation performing them more frequently is not sealing deeply enough; its contracts cannot withstand change because they were not designed to. An organisation never performing them may be ossifying, with the system drifting from its purpose but nobody willing to face the ceremony of a formal bump.

The metric is informative only across long time horizons. An organisation in its first three years of SEA adoption should not expect any MVBs and should not be alarmed by their absence.

The composite portfolio view

No single portfolio indicator is sufficient on its own. A healthy portfolio shows:

- Balanced contract age distribution

- Small, stable drift incident count with decreasing severity mix
- Extension approval rate in the 75-90% range with timely decisions
- Successful succession events that do not degrade contract integrity
- Parallel-live periods resolving within a year
- Zero or near-zero orphan contracts
- Rare MVBs when they happen, executed completely when they do

The Sealed Review Board reviews all seven indicators quarterly. Trending dashboards are valuable. Absolute values at a point in time are less informative than trends over multiple quarters.

4. Governance Health Indicators

The first two measurement layers track artefacts. The third tracks the protocols that govern those artefacts. Governance health matters because an organisation can have healthy contracts and a healthy portfolio while the protocols themselves are slowly becoming ceremonial — meetings happen but produce no decisions, reviews are signed off without substance, succession transfers are informal. When that happens, the underlying architectural health will decline eventually; governance health is the leading indicator.

Six indicators track the protocols themselves.

4.1 Quarterly Review Attendance and Substance

Attendance is easy to measure. Substance is harder but more important. Attendance shows whether people are showing up; substance shows whether decisions are being made.

The attendance component tracks: what percentage of Sealed Review Board members attended the last quarterly meeting, what percentage of Contract Owners for contracts on the agenda were present, whether the meeting occurred on schedule or was deferred.

The substance component is qualitative but important: did the meeting produce decisions recorded in writing, did any contract undergo a status change as a result, were remediation actions assigned with named owners and deadlines. A meeting that produces no recorded decisions is theatre regardless of how well attended it was.

4.2 Extension Review Turnaround Time

From the moment an extension proposal is submitted to the moment a decision is recorded, how long elapses? This is a direct measure of whether governance is operating at a sustainable pace.

The healthy range depends on the organisation's scale and the scope of the extension. For routine extensions to mid-importance contracts, 3–7 working days is typical. For extensions to high-stakes contracts (core identity, authentication, core business logic), 7–14 days may be appropriate. For extensions that cross governance boundaries (multiple Sealed Review Boards), 14–30 days can be legitimate.

The pathology to watch for is bimodal distribution: either very fast (1 day) or very slow (over 30 days), with nothing in between. Bimodal distributions typically mean that simple extensions are being waved through without substantive review while complex ones are blocked indefinitely. The healthier pattern is a unimodal distribution centred on the organisation's declared norm, with a long tail for genuinely complex cases.

4.3 Open Drift Incident Count by Severity

Drift incidents, once opened, need to be closed. An incident in state Open is one where the drift has been detected but not yet resolved. Tracking the count by severity tells you both the current state of the architecture and how quickly the organisation is working through its accumulated drift.

Severity follows the classification system specified in Section 14 (Critical, High, Medium, Low). A healthy organisation will have a small number of Low-severity incidents open at any time, very few Medium-severity ones, rare High-severity ones, and zero Critical. An organisation where Critical-severity incidents remain open for more than one sprint is one whose governance is not functioning — Critical means the drift could cause concrete harm, and leaving it open means the organisation has accepted that risk consciously or unconsciously.

4.4 Intent Document Update Frequency

Intent documents are append-only and should be updated whenever a material decision affects the contract. The update frequency measures whether this is actually happening.

For a contract that has seen no extensions or reviews, zero updates in a quarter is appropriate. For a contract that has undergone an extension in the quarter, the intent document should have been updated to capture the rationale for the extension — if it was not, either the extension was not actually rationalised or the update discipline is failing. For a contract that has been the subject of quarterly review, the intent document should reflect any discussion of shift in assumptions, emerging risk, or changed context.

The metric reveals a common failure pattern: the intent document is written once at the time the contract is sealed, and never updated thereafter. This is the beginning of Amnesia — the failure mode where the system's original intent becomes irretrievable because the documentation froze at sealing and the reasoning evolved separately in people's heads.

4.5 Board Member Rotation and Term Completion

Sealed Review Board membership is explicitly structured with rotating staggered terms. The metric tracks whether rotation is happening on schedule and whether Board members complete their terms.

An organisation where Board members routinely extend their terms, or where the same people serve for many years, has allowed the Board to become an oligarchy. Architectural decisions become identified with specific individuals rather than with the Board as an institution. Over time, the Board's authority becomes contingent on those individuals staying in the role. This is a slow failure mode but a real one.

The metric also tracks completion: did the last cohort finish their terms, or did they rotate off early? Early rotation often signals overload — the Board is asking more of members than the role description accounts for.

4.6 Governance-to-Engineering Ratio

Of the total effort spent on sealed contracts and their ecosystem, what percentage goes to governance activities (reviews, succession, audits) versus engineering activities (building, extending, verifying)?

A healthy ratio is approximately 15–25% governance, 75–85% engineering. Below 15% governance means the protocols are underpowered; the contracts may look sealed but are not being maintained as sealed. Above 25% means governance has become bureaucracy; the ratio of ceremony to delivery is unsustainable and engineers will begin routing around the governance system to get work done.

This metric is hard to calculate exactly but valuable to approximate. Organisations should use engineering time tracking, meeting schedules, and document-history timestamps to produce a rough estimate at least annually.

5. Leading vs Lagging Indicators

A subtle but operationally important distinction in SEA measurement is the difference between leading and lagging indicators. Both have value. Confusing them is one of the most common measurement mistakes.

5.1 The Distinction

A lagging indicator reports on a state that has already occurred. A drift incident, once detected and logged, is a lagging indicator — the drift has already happened. Counting drift incidents per quarter tells you about your recent past, not your future.

A leading indicator reports on conditions that are likely to produce a future state. A contract whose intent document has not been updated in three quarters despite two extensions is a leading indicator of future Amnesia — the amnesia has not happened yet, but the conditions that produce it are accumulating.

Lagging indicators tell you what happened. Leading indicators tell you what is about to happen. Good measurement requires both.

5.2 Classifying the Metrics

Metric	Classification	What it tells you
Contract Integrity Score	Lagging (mostly)	Current state of a specific contract
Drift incidents per quarter	Lagging	Drift that has already manifested
Extension approval rate	Lagging	How the review process has been running
Extension turnaround time	Leading (partly)	Decreasing times suggest governance can handle more velocity
Intent document update frequency	Leading	Future Amnesia risk
Orphan contract count	Leading	Future Ghost Ownership risk
Parallel-live duration distribution	Leading	Future Decommissioning Paralysis risk
Governance-to-engineering ratio	Leading	Future theatre or bureaucracy risk
Board member rotation	Leading	Future governance ossification risk

5.3 Why This Matters Operationally

A Sealed Review Board reviewing only lagging indicators will always be discussing past problems. A Board reviewing only leading indicators will be discussing hypothetical risks that may never materialise. Good governance requires both — lagging indicators to maintain accountability for what has happened, leading indicators to direct attention to where intervention would be most valuable.

A useful organisational principle: lagging indicators drive retrospective action (what should we do about what happened), leading indicators drive preventive action (what should we do so that a thing does not happen). Both are needed. Either alone produces either constant reaction or constant speculation.

6. Anti-Metrics

Any measurement system will be gamed. Engineers, managers, and executives optimise for what is measured. This is not a character flaw; it is an inevitable consequence of incentive structures. The measurement system itself must be designed with this in mind.

Specifically, SEA measurement should exclude several metrics that might seem natural but that actively encourage the wrong behaviours. These are anti-metrics: numbers that look like health indicators but actually measure activity in ways that incentivise dysfunction.

6.1 Contract Count

It is tempting to count the number of sealed contracts in the portfolio and treat more as better. This is wrong. A portfolio with fewer, better-chosen sealed contracts is healthier than one with many marginal ones. Rewarding contract count incentivises sealing things that should not be sealed — sealing transient concerns, sealing implementation details, sealing prematurely. The result is a portfolio full of noise that obscures the signal of which contracts actually matter.

If an organisation must publish a contract count number, it should be accompanied by a health distribution (what fraction of contracts are scoring above 75 on the Integrity Score) so that growth in count cannot disguise growth in unhealthy contracts.

6.2 Lines of Code Under Sealed Governance

Some organisations will want to measure 'how much of our codebase is governed by SEA' in terms of lines of code. This is doubly wrong. First, sealed contracts are not code — they are specifications. Counting the lines of code that implement a sealed contract mixes the durable layer with the transient. Second, line count has never been a meaningful unit for architectural quality; it is even less meaningful when applied to governance scope.

The correct question is not 'how much code is sealed' but 'which architectural boundaries are sealed,' and that question is answered by the portfolio-level indicators, not by a line-count number.

6.3 Number of Governance Meetings Held

Meeting frequency is not a measure of governance effectiveness. A quarterly review that happens reliably every 90 days and produces clear decisions is superior to weekly reviews that produce no decisions. Measuring meeting count incentivises meetings; it does not incentivise outcomes.

If meeting data is tracked at all, it should be accompanied by decision-count: how many decisions were recorded as a result of the meeting. Meetings without decisions are a warning sign, not an achievement.

6.4 Number of Extension Proposals Approved

Counting approvals incentivises approval. If a Contract Owner is measured on how many extensions they approve per quarter, they will approve more — including ones that should have been rejected or returned for rework.

The healthier view is the approval rate discussed in Section 3.3: of the proposals submitted, what fraction are approved, and is that fraction in the healthy band. Raw approval count is meaningless without context; it can go up because the architecture is healthy, or because the review process has degraded into rubber-stamping.

6.5 Time to Seal a New Contract

Organisations sometimes try to measure how quickly new contracts can be sealed, treating faster as better. This incentivises speed over quality. A contract that is sealed quickly without a proper intent document, without consideration of rejected alternatives, without careful invariant specification, will score poorly on the Integrity Score and will require remediation later — for which the remediation effort is often longer than the time saved.

Sealing a contract thoroughly takes time. The measurement that matters is not how long it takes but how well the sealed contract scores one quarter after sealing. A well-sealed contract starts at 85+ on the Integrity Score. A rushed one may start at 60 and never recover.

6.6 Adoption Count

The number of teams or divisions that have adopted SEA is not a measure of SEA's effectiveness. Organisations sometimes declare adoption based on nothing more than a team having read the whitepaper and scheduled their first review meeting. True adoption requires operating the governance protocols, maintaining the Integrity Score portfolio, and having completed at least one extension cycle. Count these; do not count mere declarations.

The meta-principle

Measure outcomes, not activity. Measure states, not rates. Measure quality, not quantity.

SEA adoption will be judged, ultimately, by whether systems adopted under it last longer and evolve more gracefully than comparable systems. That is the only metric that actually matters — and it is a metric that takes years to resolve.

The metrics in Sections 2 through 5 are proxies for that ultimate metric. Use them to guide attention and action. Do not confuse them for the real thing.

PART II

Enforcement

What physically prevents modification of a sealed contract? How is the seal actually held?

7. What Enforcement Means in Practice

The SEA 1.0 whitepaper described sealed contracts as cannot-be-modified-once-committed. This is the correct philosophical position. But a common question from implementers follows immediately: what physically prevents modification? If the only thing stopping a determined engineer from editing a sealed contract is a rule in a governance document, is the contract actually sealed, or merely labelled as sealed?

The honest answer is that in practice, sealing is enforced through a layered defence. No single mechanism is sufficient. Each layer addresses a different kind of threat to the seal, and the combination produces a practical guarantee that is strong enough for enterprise use.

7.1 The Four Layers of Enforcement

Enforcement operates at four layers, each addressing a specific gap:

Layer	What it prevents	Mechanism
Repository	Casual modification; accidental edits by engineers who do not know the contract is sealed	Git branch protection, CODEOWNERS files, required reviews, commit signing
Build pipeline	Modifications that slipped past the repository layer; modifications by automated tools	Contract hash verification at build time, merge gates, sealed-contract-test suite
Runtime	Implementations that drift from the contract over time even when the contract file itself is intact	Runtime assertions, contract conformance tests, dependency version locking
Governance	Deliberate modification by those with repository access; social pressure to bypass protocol	Sealed Review Board authority, audit trails, succession protocols, organisational culture

Each layer catches a different class of modification attempt. Casual edits hit the repository layer and stop. Modifications that slip past repository controls (perhaps through a branch that bypasses the protection rules) hit the build layer. Implementations that drift while the contract itself is untouched are caught at runtime. And deliberate governance violations — the hardest to prevent technically — are caught through audit and social-organisational mechanisms.

7.2 The Residual Risk

Even with all four layers in place, a residual risk remains: a sufficiently senior engineer with sufficient authority can override any of the technical controls. A CTO with merge permissions can approve their own changes. A Sealed Review Board could vote to retire a contract unsoundly. Technical enforcement cannot prevent legitimate authority from exercising itself.

This is not a weakness of SEA. It is an acknowledgement that in human organisations, the highest level of enforcement is always social and cultural, not technical. Technical controls make casual violations difficult and unintended violations nearly impossible, but they cannot prevent an organisation from deciding, deliberately and with authority, to break its own rules.

The purpose of enforcement is not to prevent all possible violations. It is to ensure that violations are deliberate, visible, and attributable. A violation that must be deliberately authorised, that leaves an audit trail, and that can be traced to a specific authoriser is a violation the organisation can learn from. A violation that happens silently and cannot be attributed is one that will happen again.

*Enforcement is not about making violation impossible. It is about making
silent violation impossible.*

8. Repository-Level Enforcement

The repository is the first line of defence. Most engineers interact with sealed contracts through the version control system, and most casual modifications happen because an engineer opens the wrong file and makes a well-meaning change. Repository-level enforcement catches these.

8.1 Branch Protection Rules

The branch containing sealed contracts — typically `main` or `trunk` — must be protected. The specific protections that matter for SEA:

- Direct pushes are forbidden; all changes must come through pull requests
- At least one review approval is required before a pull request can merge
- For sealed contract files specifically, the required reviewer must be the named Contract Owner or Reviewer (enforced via CODEOWNERS)
- Force-pushes are forbidden; history is immutable
- Status checks (CI pipeline, verifier, hash validation) must pass before merge is permitted

These protections exist in all major version control systems — GitHub, GitLab, Bitbucket, Azure DevOps all support them. The specific configuration varies but the principles are portable.

8.2 CODEOWNERS

A CODEOWNERS file maps paths in the repository to required reviewers. For a repository containing sealed contracts, the CODEOWNERS file should include:

```
# CODEOWNERS for sealed contract enforcement

# The contracts directory is governed by the Platform Team and
# individual Contract Owners.

# All sealed contracts require Platform Team review as a minimum
/contracts/ @platform-team

# Individual contracts require their specific Owner
/contracts/auth/ @security-architect @platform-team
/contracts/identity/ @identity-owner @platform-team
/contracts/billing/ @billing-owner @platform-team

# Intent documents are co-owned
/intent/ @platform-team

# The governance charter is the most tightly protected
/governance/charter.md @sealed-review-board

# Verifier configurations – Platform Team only
/verifiers/ @platform-team
```

The pattern: every path that contains durable artefacts has named owners who must approve changes. This is not a courtesy; without CODEOWNERS enforcement, branch protection alone would allow any reviewer to approve any change, defeating the Contract Owner's authority over their specific contract.

8.3 Commit Signing

Commits that modify sealed contracts should be cryptographically signed. This achieves two purposes. First, it proves that the commit was authored by the named author, not by someone impersonating them. Second, it creates a non-repudiable record — an authored change cannot later be disclaimed.

Commit signing requires per-engineer GPG or SSH keys registered with the version control system. The overhead is small once configured. The guarantee is significant: no one can modify a sealed contract without leaving a cryptographically verifiable trail back to themselves.

8.4 Tag Immutability for Contract Versions

When a contract is sealed at version N, a Git tag records the exact state at that moment. The tag should be immutable — once set, it cannot be moved or deleted. Tag immutability means that the sealed version is a permanent reference point, even if the branch continues to evolve with extensions and new versions.

Most Git hosts support tag protection. The SEA-specific pattern is to tag each contract version as ``seal/<contract-id>/v<version>``. For example, ``seal/auth.user_session/v1`` points to the exact commit where ``auth.user_session.v1`` was sealed. Future auditors can retrieve this state regardless of subsequent changes.

8.5 Repository Layout Conventions

Enforcement is easier when the repository layout itself reflects the sealed/transient distinction. A recommended layout:

```

repository/
├── contracts/                                # Sealed contracts (durable)
│   ├── auth/
│   │   └── user-session.v1.yaml
│   ├── identity/
│   └── billing/
├── intent/                                  # Intent documents (durable)
│   ├── auth/
│   │   └── user-session-v1.md
│   ├── identity/
│   └── billing/
├── governance/                              # Governance charter (durable)
│   ├── charter.md
│   ├── srb-members.yaml
│   └── review-schedule.md
├── verifiers/                               # Verification tooling (transient)
│   ├── kotlin/
│   ├── python/
│   └── typescript/
└── implementations/                         # Transient layer

```

```
|  └─ services/  
|  └─ libraries/  
└─ CODEOWNERS                                # Enforcement mapping
```

The separation makes it visually clear which parts of the repository are durable (contracts, intent, governance) and which are transient (verifiers, implementations). New engineers looking at the structure understand the distinction without needing to read documentation first. The structure itself communicates the architecture.

9. Build-Pipeline Enforcement

Repository-level enforcement catches most modification attempts before they reach the main branch. Build-pipeline enforcement catches what slips past, and also catches a different class of problem: implementations that no longer conform to their sealed contracts, even when the contract files themselves are intact.

9.1 The Verifier Step

Every build pipeline for a project that contains sealed contracts should include a verifier step. The verifier reads each sealed contract in the repository, checks the implementation code against the contract's interface and invariants, and returns a conformance result. If conformance fails, the build fails.

The verifier step must run before any deployment-related steps. A build that fails verification cannot be deployed, even if the engineer has administrative permission to override other checks. This is the primary technical enforcement of the sealed character of contracts: implementations that drift from their sealed contracts cannot reach production.

9.2 Contract Hash Verification

A subtler form of enforcement: every sealed contract has a cryptographic hash computed at sealing time. The hash is stored alongside the contract and is part of the sealed-tag's content. At build time, the current contract file's hash is recomputed and compared to the stored hash.

If the hashes match, the contract has not been modified since sealing. If they do not match, the contract has been modified — either legitimately (via an extension that updates the hash and creates a new sealed tag) or illegitimately (via a direct edit that bypasses the extension protocol). The build fails, and the discrepancy is investigated.

The value of hash verification is that it catches ungoverned modifications even when those modifications were made by someone with repository permissions. The branch protection layer trusts the reviewer's judgement; the hash verification layer trusts only the cryptographic evidence. Someone who could merge an ungoverned change directly would still fail the hash check, because the hash in the sealed tag would no longer match the current file.

9.3 Merge Gates

A pull request modifying a sealed contract file must pass additional gates beyond normal code review. The recommended gates:

8. The contract's Owner and Reviewer have both approved (enforced via CODEOWNERS and branch protection)
9. The change references an approved extension proposal or a formal retirement authorisation
10. The intent document has been updated in the same pull request (this prevents contracts and intent documents from drifting apart)
11. The verifier step passes with the new contract version
12. The change history in the contract YAML has been updated to reflect the change

Each gate is small, but the combination is strong. A change that omits any element — an approver missing, no extension proposal referenced, intent not updated, verifier failing, or change history not recorded — is rejected. The protocol cannot be partially followed; either all elements are present or the change does not merge.

9.4 Sealed Contract Test Suite

In addition to the verifier, many organisations benefit from a dedicated test suite that asserts sealed contract properties at build time. Examples:

- A test that fails if any sealed contract lacks an associated intent document
- A test that fails if any sealed contract has a status of 'retired' but is still referenced by live implementations
- A test that fails if any invariant declared in a sealed contract has no corresponding test or assertion in the codebase
- A test that fails if the CODEOWNERS file does not properly map each sealed contract to its named Owner

These tests are specifications-about-specifications. They assert properties of the sealed contract portfolio itself, not properties of the implementations. They run alongside normal tests and catch structural drift in the portfolio the same way normal tests catch functional regressions in the code.

9.5 Incremental vs Full Verification

For large portfolios, running the full verifier against every contract on every build becomes slow. Two strategies mitigate this:

Incremental verification

On each pull request, run the verifier only against contracts whose implementations were modified or whose contract files were touched. Run the full verifier on merges to main and on scheduled nightly builds. This gives pull request feedback fast and catches cross-contract drift on a regular cadence.

Caching with hash invalidation

Verifier results can be cached keyed on (contract-hash, implementation-hash). If neither has changed since the last successful verification, the cached PASS can be reused. When either changes, the cache entry is invalidated and verification runs freshly. Typical cache hit rates exceed 80% for mature portfolios, dramatically reducing verification time.

The verifier is infrastructure, not optional

A SEA adoption that does not have a working verifier integrated into the build pipeline is not fully adopted. The governance protocols can catch many drift scenarios but cannot catch implementation drift during active development.

Building the verifier is not optional work. It is the single most important piece of enforcement infrastructure. Organisations should prioritise a working verifier — even a basic one — before expanding to additional contracts or additional teams.

Verifier work is typically 2-4 weeks for the first working version in a given stack, with ongoing maintenance measured in days per quarter.

10. Registry Patterns

Where do sealed contracts physically live? This question has two reasonable answers, depending on the organisation's scale and structure. Each pattern has different implications for governance, tooling, and cross-team coordination.

10.1 Centralised Registry Pattern

All sealed contracts across the organisation live in a single dedicated repository. Implementations live elsewhere — typically in per-team or per-service repositories — and reference the contracts by their `seal_id`.

Advantages of the centralised pattern:

- Single source of truth. A team looking for a contract knows exactly where to find it.
- Unified governance. The Sealed Review Board can review the entire portfolio from one repository.
- Easier reporting. Portfolio-level metrics are straightforward to calculate when all contracts are in one place.
- Simpler tooling. The verifier infrastructure needs to know only one contract location.

Disadvantages:

- Can become a bottleneck. Every change requires coordination with the central repository's access controls.
- Feels remote to individual teams. The contract governing a team's service lives outside the team's own repository, which can reduce the team's sense of ownership.
- Single point of failure for governance. Repository outage or corruption affects the entire organisation.

The centralised pattern fits organisations with fewer than approximately 500 engineers, or organisations with strong central architecture functions regardless of size. It is the simpler pattern and should be the default unless a reason exists to distribute.

10.2 Federated Registry Pattern

Sealed contracts live alongside the services or components they govern. A service repository contains both the service's implementation and its sealed contracts. A central registry exists but is a manifest rather than the actual contracts — it lists where each contract lives and maintains portfolio-level metadata.

Advantages of the federated pattern:

- Locality. The contract lives next to the code it governs. Engineers editing the code see the contract.
- Team ownership. Teams feel genuine ownership of their contracts because they live in the team's own repository.
- Resilience. No single repository outage affects all contracts.
- Scales to very large organisations. The central manifest stays manageable even when there are thousands of contracts across hundreds of repositories.

Disadvantages:

- Coordination overhead. Portfolio-level operations require crossing many repositories.

- Consistency challenges. Contract format standards, CODEOWNERS patterns, and verifier configurations must be propagated across all repositories.
- Harder onboarding. New engineers must learn to discover contracts rather than browse them in one place.

The federated pattern fits organisations with more than 500 engineers or strong team autonomy cultures. It requires more mature tooling — a central manifest, cross-repository search, consistent templates — but scales to very large organisations better than the centralised pattern can.

10.3 The Central Manifest

In both patterns, a central manifest is valuable. In the centralised pattern, the manifest is an index of the single contracts repository. In the federated pattern, the manifest is a pointer index to contracts across many repositories. Either way, the manifest provides:

```
# registry-manifest.yaml – central SEA manifest

organization: Example Corp
last_updated: 2026-04-22
total_sealed_contracts: 47
total_governance_bodies: 4

contracts:
  - seal_id: auth.user_session.v1
    location: https://github.com/example/platform-contracts/contracts/auth/user-session.v1.yaml
    status: sealed
    governance_body: Platform-SRB
    current_owner: security-architect@example.com
    integrity_score: 87
    last_reviewed: 2026-03-15

  - seal_id: identity.user.v2
    location: https://github.com/example/platform-contracts/contracts/identity/user.v2.yaml
    status: sealed
    governance_body: Platform-SRB
    current_owner: identity-owner@example.com
    integrity_score: 91
    last_reviewed: 2026-03-15

  - seal_id: billing.invoice.v1
    location: https://github.com/example/billing-service/contracts/invoice.v1.yaml
    status: sealed
    governance_body: Billing-SRB
    current_owner: billing-owner@example.com
    integrity_score: 72
    last_reviewed: 2026-02-28
```

The manifest is itself a sealed artefact — it is the registry's metadata, and organisations should treat changes to it with appropriate governance. Most manifests are generated rather than hand-edited; tooling walks the repository structure, computes current Integrity Scores, and updates the manifest on a schedule.

11. Breach Response

What happens when enforcement fails? Despite the four layers, a breach is possible — an ungoverned modification that reaches production, a sealed contract that was silently edited and the modification was missed. Every organisation adopting SEA should have a breach response protocol before they need it.

11.1 Detection

Breaches are detected in several ways:

- Hash mismatch during build — the technical layer caught it, build failed
- Verifier failure during build — the implementation and contract have drifted
- Audit review surfaced a discrepancy — the quarterly review found a change that was not properly governed
- Production incident traced to a contract violation — the breach caused operational harm
- External report — a consumer, auditor, or partner noticed the violation

The detection mode matters. A breach caught by the build pipeline is minor (it was caught before reaching production). A breach traced to a production incident is severe (the breach caused actual harm). The response should be proportional to the severity.

11.2 Immediate Response

When a breach is detected, the immediate response has four steps, in this order:

13. Contain. Stop further deployments of the affected code until the breach is understood. In the build-pipeline case this is automatic — the build has already failed. In the production-incident case, it may require rolling back the affected component.
14. Investigate. Identify how the breach occurred. Was the sealed contract modified without authorisation? Was an implementation deployed that did not match the contract? Was the verifier bypassed? Answer the how before discussing the who.
15. Contain — stop further deployments of the affected code until the breach is understood.
16. Investigate — identify how the breach occurred before discussing who caused it.
17. Document — open a Drift Incident record (CD-series identifier). Classify severity. Assign an investigator.
18. Notify — inform the Sealed Review Board and the Contract Owner. Follow the organisation's incident protocol for production-affecting breaches.

11.3 Root-Cause Classification

Every breach has a root cause. Classifying the cause correctly determines the response:

Root cause	What it means	Appropriate response
Tooling gap	An enforcement mechanism failed. Branch protection was mis-configured, verifier crashed, hash check was skipped.	Fix the tooling. The protocol was correct; the infrastructure failed.
Governance gap	The protocol allowed the breach through legitimate but insufficient steps.	Strengthen the protocol. The infrastructure worked; the rules did not

Root cause	What it means	Appropriate response
	Approvals were given without proper review.	catch the case.
Education gap	The author did not know the contract was sealed or what the protocol required.	Train the team. The infrastructure and rules were correct; the human knowledge failed.
Deliberate bypass	Someone knowingly circumvented the protocol.	Governance review. The infrastructure, rules, and knowledge were all in place; the failure was in authority or discipline.

Mis-classifying the root cause produces the wrong response. Treating a deliberate bypass as an education gap means more training that does not address the actual problem. Treating a tooling gap as a governance gap means adding rules on top of broken infrastructure that the rules also cannot fix. Honest root-cause classification is essential.

11.4 The Drift Incident Log

All drift incidents are recorded in a formal log. The log is append-only — entries may be updated with status changes and resolution notes, but never deleted. The log is one of the durable artefacts of SEA; it is how the organisation learns.

Each incident record contains:

- Unique identifier (CD-YYYY-NNN)
- Date detected and date occurred (these may differ significantly)
- Detection mode (from the list in 11.1)
- Affected contract(s) and implementation(s)
- Severity classification (Critical, High, Medium, Low)
- Root cause classification (from the four types in 11.3)
- Investigation findings
- Remediation applied
- Preventive action taken so the same class of incident does not recur
- Status (open, resolved, accepted-risk)

11.5 Learning From Breaches

The most important use of the drift incident log is organisational learning. Each breach, once resolved, becomes data that informs future decisions. The quarterly Sealed Review Board meeting should include a review of the incidents closed during the quarter: what patterns emerged, which contracts were affected most often, which root causes were most common.

Over time, this learning shapes the architecture. Contracts that are breached repeatedly are candidates for redesign — the breaches suggest the contract is not well aligned with how engineers think about the domain. Root causes that recur are candidates for tooling investment. The log becomes the organisation's memory of where its architectural discipline is weakest and where investment will produce the most improvement.

A breach that produces no learning is a breach that will happen again,

with interest.

PART III

Verification

Beyond PASS/FAIL. Richer compliance vocabulary, severity classification, static and runtime mechanisms working together.

12. The Verification Layer, Expanded

The SEA 1.0 whitepaper described the verifier as returning PASS or FAIL. This is the simplest possible model. In practice, verification in enterprise systems is more nuanced. Implementations are often partly conforming — most of the contract is satisfied, but a specific element is not. Some violations are catastrophic; others are minor and can be addressed in a regular release cycle. And the relevant verification often requires checking not only the static code but the runtime behaviour against the contract's runtime-visible invariants.

This part expands the verification layer with three refinements: five compliance states instead of two, severity classification for violations, and the complementary use of static and runtime verification mechanisms.

12.1 Why PASS/FAIL Is Insufficient

A binary model forces all non-conforming cases into a single bucket labelled 'failed.' But not all failures are equivalent. Consider three cases:

19. An implementation is missing a declared operation entirely. The contract specifies ``create_session`` and ``revoke_session``; the implementation only contains ``create_session``. Consumers of the contract will fail when they call ``revoke_session``. This is a blocking violation.
20. An implementation satisfies all operations and invariants but is missing an explicit test for invariant INV-2. The invariant holds in practice (the code satisfies it) but there is no automated assertion that would detect future violation. This is a warning, not a blocker.
21. An implementation has introduced an extension that is technically within the extension policy but was added without following the extension review protocol. The extension is acceptable; the process was not. This is a governance violation, not an implementation violation.

Treating all three cases as 'FAIL' is technically accurate but operationally useless. The response to each is different, and the urgency of the response is different. A richer vocabulary allows the verifier to communicate what kind of non-conformance exists, so that the appropriate response can be taken.

13. Five Compliance States

The verifier should classify each contract-implementation pair into one of five states. The states form a hierarchy from fully healthy to fundamentally broken.

13.1 State: Conforming

All operations declared in the contract are present in the implementation. All invariants have corresponding enforcement (tests, assertions, or static checks). The implementation's public surface matches the contract exactly, with no undeclared operations and no missing ones. No extension has been applied that was not governed through the extension protocol.

This is the target state. An implementation in Conforming state needs no action.

13.2 State: Conforming With Warnings

All operations are present. All invariants are satisfied in practice. But one or more enforcement mechanisms are missing — an invariant lacks an explicit test, or the implementation has added public operations that are not declared in the contract but that do not violate the extension policy (they are legitimate extensions that simply have not been formally proposed yet).

An implementation in this state passes the build but the warnings are recorded. The warnings should be addressed in the next regular cycle — either by adding the missing enforcement or by formally proposing the undeclared extensions. Warnings that persist across multiple verification cycles are a signal that governance discipline is slipping.

13.3 State: Partial

Some operations declared in the contract are missing from the implementation, or some invariants are provably violated in specific code paths. The implementation is not completely broken, but consumers relying on the missing operations or the violated invariants will experience failures.

This state blocks production deployment. The build may succeed if the build is targeted at a component that does not use the missing pieces, but any deployment that would expose the partial surface to consumers must be blocked. Partial state requires immediate remediation: either the missing operations are added, the invariant violations are fixed, or the contract is formally amended to match the implementation's reduced scope.

13.4 State: Non-Conforming

The implementation substantially diverges from the contract. Many operations are missing or have different signatures. Multiple invariants are violated. The implementation appears to have been written without reference to the contract, or the contract has evolved significantly while the implementation has not.

Non-conforming state is a serious governance failure. Either the implementation has been allowed to drift without enforcement, or the contract has been modified without the implementation being updated. Deployment is blocked; a full review of the contract-implementation pair is required before the situation can be resolved.

Non-conforming state should never be reached in a properly enforced SEA deployment. If it is reached, the enforcement infrastructure has failed at multiple layers simultaneously.

13.5 State: Unverifiable

The verifier cannot determine conformance. This may happen because the contract is malformed (invalid YAML, missing required fields), because the implementation is in a state the verifier cannot analyse (for instance, heavy use of dynamic language features that defeat static analysis), or because the verifier itself has a bug or configuration problem.

Unverifiable state is not the same as non-conforming. A non-conforming implementation has been analysed and found wanting; an unverifiable one could not be analysed. The appropriate response is different: unverifiable state requires investigation into why verification failed, not remediation of the implementation. Sometimes the resolution is fixing the contract; sometimes it is extending the verifier; occasionally it is accepting that a particular contract is not amenable to automated verification and relying on manual review for that specific case.

13.6 State Transitions and Reporting

The verifier reports the state for each contract-implementation pair on every run. Over time, transitions between states tell an important story:

Transition	What it signals
Conforming → Conforming with Warnings	Enforcement discipline is slipping; attention needed before it worsens
Conforming with Warnings → Partial	Significant governance gap; root cause investigation needed
Partial → Non-Conforming	The drift is expanding; full review required
Non-Conforming → Partial → Warnings → Conforming	Active remediation; confirm each step is intentional
Any → Unverifiable	Verifier or contract problem; infrastructure issue, not governance issue

Dashboards that show the state distribution across the portfolio — percentage of contracts in each state — give the Sealed Review Board a quick picture of overall health. A healthy portfolio has 80%+ in Conforming, 15-20% in Conforming with Warnings, and near-zero in the other states. A portfolio with significant Partial or Non-Conforming percentages is one where enforcement has systematically failed and immediate intervention is warranted.

14. Severity Classification for Violations

Within the non-conforming states (Partial and Non-Conforming), individual violations have different severity. A missing optional field is not the same as a violated security invariant, even though both are technically 'non-conforming.' Severity classification allows the verifier to communicate not just that something is wrong but how urgent the wrongness is.

14.1 The Four Severity Levels

Severity	Definition	Response timing
Critical	The violation could cause immediate operational harm, security breach, data loss, or regulatory non-compliance if deployed. Examples: an authentication invariant violated, a data retention rule violated, a mandatory audit trail missing.	Deployment blocked. Emergency remediation. Incident declared.
High	The violation will cause visible operational problems, failed deployments, or incorrect business outcomes within 30 days. Examples: a missing operation that consumers rely on, a schema mismatch that breaks integration.	Deployment blocked. Remediation within current sprint.
Medium	The violation creates confusion, slows onboarding, or increases risk of future problems without immediate failure. Examples: documentation drift, missing invariant tests, undeclared extensions that are within policy.	Deployment allowed with warnings. Remediation within current quarter.
Low	Cosmetic or stylistic inconsistencies with no operational impact. Examples: naming convention drift, non-material formatting differences.	Deployment allowed. Remediation opportunistic or deferred.

14.2 Classifying in Practice

The verifier classifies each detected violation. The classification is based on the invariant or rule being violated, not on the subjective judgement of a specific moment. This is important: severity should be reproducible across runs of the verifier, not vary based on who is reviewing or when.

Each invariant in a sealed contract is tagged with a severity level in the contract itself. The tag defines what severity any violation of that invariant carries. For example:

```
invariants:
- id: INV-1
  statement: Session tokens are opaque to clients
  rationale: Exposing user identifier in the token permits lateral
             account compromise via token inspection.
  violation_severity: Critical

- id: INV-2
  statement: Session revocation is atomic
  rationale: Partial revocation leaves stale sessions that appear
             valid to downstream services.
  violation_severity: Critical
```



```

- id: INV-3
  statement: Created sessions log an audit event
  rationale: Compliance requirement under SOC 2.
  violation_severity: High

```

With severities declared at the invariant level, the verifier reports violations with their proper severity automatically. The classification is not guessed at the time of verification; it is declared at the time of sealing, and then applied consistently whenever verification runs.

14.3 Cumulative Severity Rules

A pattern of multiple lower-severity violations can collectively constitute a higher-severity concern. Two useful rules:

22. Three Medium violations in the same subsystem within a single verification cycle escalate to one High finding about the subsystem. The escalation reflects the pattern: if the subsystem has multiple independent drift points, the subsystem as a whole is drifting.
23. Five Low violations of the same type across the portfolio escalate to one Medium finding about the type. The escalation reflects that a repeated low-severity pattern is usually a root-cause problem that deserves focused attention, not five separate minor fixes.

Cumulative severity prevents organisations from rationalising multiple small violations as individually acceptable while the portfolio silently degrades. The pattern matters as much as the individual cases.

15. Static vs Runtime Verification

Verification happens at two distinct moments: at build time (static verification) and during execution (runtime verification). Each catches a different class of violation. A comprehensive verification strategy uses both, with each chosen for what it is best at.

15.1 Static Verification

Static verification runs at build time, without executing the code. It inspects the source code, the compiled artefacts, the schema definitions, and the contract files, and it draws conclusions from what it can see without running anything. Examples of what static verification catches:

- Missing operations (the contract declares ``create_session``; the implementation has no function by that name)
- Signature mismatches (the contract's input types do not match the implementation's parameters)
- Structural invariants provable from types (a field declared as non-optional in the contract is declared as nullable in the implementation)
- Undeclared public surfaces (the implementation exports operations that are not in the contract)
- Hash mismatches between sealed contract and current file state

Static verification is fast and deterministic. It catches a large class of common violations. It cannot catch behavioural violations — an operation with the correct signature can still violate its postconditions at runtime.

15.2 Runtime Verification

Runtime verification runs during execution, either in test environments or in production. It observes actual behaviour and compares it against the contract's runtime-visible invariants. Examples of what runtime verification catches:

- Postcondition violations (the operation returned a value that does not satisfy the declared postcondition)
- Invariants over state (a revocation that left some tier still believing the session was valid)
- Timing invariants (an operation declared to complete within a time bound that occasionally exceeds it)
- Cross-operation invariants (two operations that should always be called in a specific order have been called out of sequence)

Runtime verification requires instrumentation. The contract's invariants are translated into assertions, guards, or logging that observes actual behaviour. The overhead in production depends on the nature of the invariants; many can be checked with minimal impact, while some require more substantial instrumentation.

15.3 The Complementary Strategy

Static and runtime verification are complementary. A typical SEA deployment uses both in combination:

Verification type	Runs when	Primary value
Static — at build	On every pull request and merge	Fast feedback to developers; catches structural violations
Static — scheduled	Nightly or weekly full portfolio scan	Catches cross-contract drift; updates the Integrity Scores
Runtime — in test	Integration test runs, CI test suite	Catches behavioural violations before they reach production
Runtime — in production	Continuous, with appropriate sampling	Catches violations that only manifest under production conditions

Each layer catches what the previous cannot. Static verification in CI is fast and catches most violations. Scheduled portfolio scans catch drift that accumulates slowly. Runtime verification in test catches behavioural violations before production. Runtime verification in production is the last line of defence — it catches what slipped past everything else.

The cost of comprehensive verification is real but manageable. The build overhead of static verification is typically measured in seconds. The runtime overhead of well-designed assertions is usually under 1%. Scheduled scans run off the critical path. The investment pays back through the drift incidents that never happen because the verification caught them first.

16. Cross-Contract Verification

Individual contract verification catches violations of a single contract. But sealed contracts interact. Contract A declares a dependency on contract B; contract C extends contract D; contract E is being replaced by contract F in a parallel-live transition. These interactions create verification concerns that single-contract verification does not address.

16.1 Dependency Verification

A contract that declares dependencies on other contracts (via the ``dependencies`` field) imposes verification obligations at the portfolio level. Specifically:

- The declared dependencies must exist in the registry
- The declared dependency version must match a sealed version of the target contract (not a retired or proposed version)
- The implementation must actually use the dependency in a way consistent with the dependency declaration
- If the dependency is later retired, the depending contract cannot remain in Conforming state; it must either update its dependency or itself be retired

Dependency verification runs at the portfolio level — typically as part of scheduled scans rather than per-pull-request verification, because it requires cross-contract analysis.

16.2 Extension Relationship Verification

When one contract extends another (via the ``supersedes`` field or an equivalent extension-relationship field), the relationship itself is a property that must be verified. The extending contract must be consistent with the extended one — it must not contradict invariants that were declared in the extended contract, and it must be within the extension policy of the extended contract.

Verification checks:

- The extended contract exists and is in Sealed status
- The extension policy of the extended contract permits the specific extension
- None of the invariants in the extended contract are weakened by the extension
- If the extension adds new invariants, they do not contradict existing ones

16.3 Parallel-Live Verification

During parallel-live transitions, two contracts coexist: the outgoing and the incoming. Verification during this period has additional concerns:

- Both contracts must remain in Conforming or Conforming with Warnings state — the parallel-live status is not an excuse for letting either decay
- Consumers must be declared as consuming one contract or the other; ambiguous consumers indicate incomplete migration
- The consumer migration progress should be measurable — what percentage have moved to the new contract
- Any invariant that differs between the two contracts must be carefully handled; consumers of the old contract should not be exposed to behaviour that would violate the old invariants, even as the new contract allows different behaviour

Parallel-live verification often requires custom tooling specific to the contracts involved. It is one of the places where per-contract verification effort is highest, because the transitions are where the most can go wrong.

16.4 Portfolio Consistency Checks

Beyond pairwise relationships, some verification operates across the full portfolio:

- No two contracts should have identical `seal_ids` — duplicate identifiers would create ambiguity in dependencies and references
- Every contract whose status is Sealed should have a corresponding sealed git tag pointing to its sealing commit
- Every declared Contract Owner should be a currently valid entity (usually an email address or employee identifier) registered with the organisation
- Every governance body referenced by any contract should exist in the governance registry

These checks are lightweight but catch a class of administrative drift that is otherwise invisible. They run as part of the manifest-update process, typically daily or on every merge to main.

Verification as a system, not a tool

The verifier is not a single program. It is a system composed of multiple checks running at multiple times, producing structured output that feeds into the Integrity Scores and the breach response protocol.

An organisation adopting SEA should plan the verification infrastructure as a system: which checks run statically, which run at runtime, which run per-pull-request, which run on schedule, how results are aggregated, where they are displayed.

Most organisations will not get this right on the first attempt. Expect the verification system to evolve over the first year of adoption as real violation patterns reveal which checks are most valuable.

PART IV

Evolution

How sealed systems change over time. Extensions, parallel-live operations, rollback, major version bumps, retirement.

17. The Evolution Layer in Depth

Of all the operational dimensions of SEA, evolution is where the most can go wrong. A contract that never changes is either ossifying (avoiding change that should happen) or irrelevant (the system has moved past it silently). A contract that changes frequently is either not sealed in the first place or being treated as if the seal is negotiable. Getting evolution right is the subtle skill that separates SEA adopted as discipline from SEA adopted as ceremony.

The SEA 1.0 whitepaper identified three evolution mechanisms: extension, parallel-live replacement, and major version bump. This part of the handbook specifies each in operational depth, adds two additional concerns that the whitepaper treated abstractly (rollback and retirement), and provides the patterns and playbooks that turn the abstract mechanisms into executable protocols.

17.1 The Evolution Hierarchy

The mechanisms form a hierarchy, from lightest ceremony to heaviest:

Mechanism	When used	Approval authority	Ceremony
Extension	Adding new capability alongside existing contracts	Contract Owner + Reviewer	Light
Parallel-Live Replacement	Replacing a contract with an improved version	Sealed Review Board	Medium
Rollback	A new contract proved flawed mid-transition; returning to the old	Sealed Review Board + executive notification	Medium-Heavy
Major Version Bump	Fundamental system purpose has changed; contracts no longer serve	Sealed Review Board + executive sponsor; unanimous SRB	Heavy
Retirement	A contract's lifetime ends; no consumers remain	Contract Owner + SRB witnesses	Light-Medium

The correct mechanism for a given situation is the lightest one that is sufficient. Using a heavier mechanism than needed wastes ceremony and slows delivery. Using a lighter mechanism than needed skips safeguards and produces hidden risk. Matching the mechanism to the situation is an act of architectural judgement, supported by clear criteria described in the sections that follow.

17.2 Evolution as a Conversation

A useful frame: evolution of sealed contracts is a conversation between the organisation and its systems. Each evolution mechanism is a way of saying something specific:

- Extension says: 'The contract is correct. We need it to do more.'
- Parallel-Live Replacement says: 'The contract is almost right. We have designed a better version and want to transition to it.'
- Rollback says: 'Our replacement was flawed. We need to return to the original while we reconsider.'
- Major Version Bump says: 'The purpose the contracts serve has changed. We need a new set of contracts for the new purpose.'

- Retirement says: 'This contract served its purpose. Its time is over. We archive it with dignity.'

Each statement is distinct. Using the wrong statement produces wrong behaviour. An extension used as a de facto replacement (because the team did not want to go through the replacement protocol) leaves the system with two contracts where one should exist. A replacement used when an extension would suffice (because the team over-scoped the change) introduces unnecessary migration work. The honest framing matters.

18. Extension Pattern Catalogue

Extensions are the most common evolution event. Most changes to a sealed system are extensions: new features, new integrations, new capabilities layered on existing contracts. This section catalogues the common shapes an extension takes, so that teams can recognise the pattern they are working with and apply the appropriate protocol.

18.1 Pattern A — Additive Output Fields

The most common extension pattern. An existing operation's output is augmented with new fields that carry additional information. The new fields are always optional, so consumers that do not know about them continue to work unchanged.

```
# Original contract
interface:
  operations:
    - name: create_session
      inputs: [user_id, device_fingerprint]
      outputs:
        - session_token: OpaqueToken
        - expires_at: Timestamp

# Extension — add optional device_trust_score to output
interface:
  operations:
    - name: create_session
      inputs: [user_id, device_fingerprint]
      outputs:
        - session_token: OpaqueToken
        - expires_at: Timestamp
        - device_trust_score: Optional<Float> # new in extension E-001
```

Pattern A is usually safe, but two checks matter. First, the new field must truly be optional — existing consumers must behave correctly whether the field is present or not. Second, the field must not encode information that existing consumers should have been consuming but were not (which would be a design error disguised as an additive extension).

18.2 Pattern B — New Capability Operations

An extension adds an entirely new operation to the contract's surface. The new operation is distinct from existing ones and does not change their behaviour. Often the new operation is implemented in terms of existing sealed operations, treating them as building blocks.

```
# Original contract
operations:
  - name: create_session
  - name: revoke_session

# Extension — add bulk revocation
operations:
```

```

- name: create_session
- name: revoke_session
- name: revoke_all_user_sessions    # new in extension E-002
  implementation_note: |
    Calls revoke_session for each active session of the user.
    Does not bypass the sealed revoke_session operation.

```

Pattern B explicitly preserves the existing sealed operations. The new operation can only do things that the existing ones already permit, composed together. This pattern is the software analogue of adding a pillar alongside existing pillars in a bridge, to support new load without modifying the original structure.

18.3 Pattern C — Invariant Strengthening

An extension adds new invariants that strengthen existing guarantees without weakening any. The contract's consumers benefit from the stronger guarantees; implementations must now satisfy more requirements but continue to satisfy all previous ones.

```

# Original invariants
invariants:
- id: INV-1
  statement: Session tokens are opaque to clients
- id: INV-2
  statement: Session revocation is atomic

# Extension – add timing guarantee
invariants:
- id: INV-1
  statement: Session tokens are opaque to clients
- id: INV-2
  statement: Session revocation is atomic
- id: INV-3    # new in extension E-003
  statement: Session revocation completes within 500ms
  rationale: Required by updated security compliance framework.

```

Pattern C is logically safe — existing consumers already behave correctly, and the new invariant does not change what they see. The caution is that implementations must be updated to satisfy the new invariant, and the extension should not be approved until implementation conformance can be demonstrated.

18.4 Pattern D — Wrapper Contract

Sometimes a new capability needs to compose existing sealed contracts in ways that are themselves worth sealing. A wrapper contract is a new sealed contract that is expressed entirely in terms of existing sealed contracts — it is a higher-level abstraction built from lower-level ones.

```

# Wrapper contract that composes existing sealed contracts
seal_id: auth.authenticated_session.v1

description: |
  Represents an authenticated session – a session created

```

```

for a user who has passed the full authentication flow.
This contract composes user_session and authentication_result.

interface:
  operations:
    - name: create_authenticated_session
      inputs: [user_id, authentication_result]
      outputs: [session_token, expires_at, auth_level]
      implementation_note: |
        Validates the authentication_result, then calls
        auth.user_session.v1:create_session.

  dependencies:
    - seal_id: auth.user_session.v1
      version: 1
    - seal_id: auth.authentication_result.v1
      version: 1

```

Pattern D is extension at the portfolio level rather than the single-contract level. It is appropriate when a domain concept is emerging that deserves first-class status. The new contract goes through the full sealing process; it is not a shortcut around normal governance.

18.5 Patterns to Recognise and Refuse

Some proposed extensions are actually replacements in disguise. Recognising these and refusing them — either by redirecting to the replacement protocol or by redesigning the proposal — is an important Owner responsibility.

Anti-pattern 1 — Signature 'clarification'

A proposal presents itself as an extension but actually changes the signature of an existing operation. Perhaps it argues that the change is semantically equivalent. The signature of a sealed operation cannot change; if clarification is truly needed, the appropriate path is parallel-live replacement, not extension.

Anti-pattern 2 — Invariant weakening

A proposal adds an 'exception' or 'edge case' to an existing invariant. For example, 'revocation is atomic except in the case of degraded backend services.' This weakens the invariant and breaks consumers who relied on the original guarantee. Extension policy typically forbids this; the proposal must be restructured or rejected.

Anti-pattern 3 — Operation semantics drift

A proposal keeps the operation's signature but changes its behaviour. For instance, `create\_session` used to return an expired token on invalid input; the proposal has it return a null value instead. The operation's contract-visible behaviour is changing, even if the signature is not. This is a replacement, not an extension.

Anti-pattern 4 — The extension avalanche

A contract receives extension after extension. Each individual extension is legitimate, but the accumulated set has effectively turned the contract into something different from what was originally sealed. When this pattern appears, the appropriate response is to pause and consider whether the

contract as a whole should undergo parallel-live replacement — consolidating the accumulated extensions into a cleaner v2.

19. The Parallel-Live Playbook

Parallel-live replacement is the mechanism for superseding a sealed contract with an improved version while maintaining service continuity. Both versions operate simultaneously during a transition period; consumers migrate to the new version on their own cadence; the old version is retired once usage reaches zero.

Parallel-live transitions are where many good intentions go to rot. A transition that stalls mid-migration produces a system with two live contracts that should have been one, and the old contract may never actually retire. This section provides the operational playbook that keeps transitions on track.

19.1 When Parallel-Live Is the Right Mechanism

Four conditions together indicate parallel-live is the appropriate choice:

24. The new contract would violate the extension policy of the old one — that is, what is being proposed is not something that can be added alongside, it is a different shape of contract
25. The change is not fundamental enough to require a Major Version Bump — the system's purpose is unchanged, only a specific contract needs to be re-specified
26. There are active consumers of the old contract who cannot migrate instantaneously — this is what makes parallel-live necessary rather than direct replacement
27. The improvement is worth the transition cost — parallel-live is expensive; if the new contract is only marginally better, extensions or simply living with the old contract may be more appropriate

19.2 The Transition Phases

A well-run parallel-live transition has seven phases:

Phase 1 — Proposal and SRB approval

The new contract is drafted with its full intent document. It explicitly references the old contract in its `supersedes` field. The SRB reviews the proposal. Approval by the SRB is required before any implementation begins; approval includes agreement on the target duration of the transition (typically 6–12 months).

Phase 2 — New contract sealing

Once approved, the new contract is sealed with status `sealed`. At this point, the old contract's status is updated to `parallel-live-with-successor` — still operational, still sealed, but now known to be on a migration path. Both contracts are in the registry; the manifest reflects their relationship.

Phase 3 — Implementation of the new contract

Production-ready implementations of the new contract are built. These may run alongside existing implementations or replace them; the key is that the new contract has working implementations before any consumer is asked to migrate. Migration cannot begin until the target is real.

Phase 4 — Consumer notification

Known consumers of the old contract are notified that a new contract exists and that migration is expected. The notification includes: the new contract's specification, the migration timeline, the support available, and the end-of-life date for the old contract. Notification is not a one-time email; it is a sustained communication effort, because consumers are distributed and may not all be listening on the first pass.

Phase 5 — Gradual migration

Consumers migrate at their own cadence, within the overall transition window. During this phase, the platform team tracks migration progress — what percentage of known consumers have moved, what blockers are preventing the remaining migrations, whether any consumers have asked for timeline extensions. The progress is reviewed at each quarterly SRB meeting.

Phase 6 — Final migration push

As the transition window approaches its end, a concerted effort identifies the remaining consumers. Some will migrate easily with a little attention; others will have genuine blockers that may require extending the window; a small number will be found to be abandoned or zombie consumers that can be disregarded. Active engagement during this phase is what prevents transitions from stalling.

Phase 7 — Retirement

Once the consumer count for the old contract reaches zero (or acceptable residual, if some abandoned consumers cannot be formally retired), the old contract enters the retirement protocol (Section 22). Its status becomes `retired`. Its specification is preserved in the archive. It is no longer a candidate for new consumers.

19.3 Typical Durations

For planning purposes, typical durations for each phase in an enterprise context:

Phase	Typical duration
1 — Proposal and SRB approval	2–6 weeks
2 — New contract sealing	1 week (administrative)
3 — Implementation of new contract	4–12 weeks depending on complexity
4 — Consumer notification	2–4 weeks (initial); ongoing
5 — Gradual migration	3–9 months
6 — Final migration push	2–6 weeks
7 — Retirement	1–2 weeks

The total is 6–15 months. Shorter transitions (under 3 months) typically indicate that parallel-live was unnecessary — the change could have been made directly. Longer transitions (over 18 months) typically indicate the transition is stalling; intervention is needed to prevent permanent parallel-live state.

19.4 Common Failure Modes and Responses

Failure mode	Response
Consumer migration is not happening because the new contract has missing features	Return to Phase 3; address the gaps before resuming migration
Migration stalls because a few consumers are unresponsive	Formal escalation to consumer management; deadline extension if legitimate; decision to abandon consumers if they cannot be reached

Failure mode	Response
New contract is discovered to have its own flaws during migration	Consider rollback (Section 20) rather than completing migration to a flawed target
Transition period exceeds agreed window	SRB review; either extend with justification or accelerate the final push
Old contract gains new consumers during transition	Investigate why — usually indicates the new contract is not discoverable or not ready; address before continuing

20. Rollback Patterns

What happens when a new contract, mid-transition, turns out to be wrong? Not subtly wrong in a way that can be patched, but fundamentally wrong — the invariants were mis-specified, the new shape does not serve the real needs, a flaw only surfaces under production load that makes the new contract untenable?

This situation is uncomfortable to plan for. Teams want to believe their replacement designs are correct. Acknowledging upfront that they might be wrong feels defeatist. But sealed contracts that are in production cannot be silently reverted, and there is no mechanism more dangerous than the one that does not exist until it is needed in a crisis.

20.1 The Rollback Principle

The principle: a parallel-live transition can be aborted. The old contract remains sealed throughout the transition, which means rollback is always available — consumers that have not yet migrated simply stay on the old contract, and consumers that have migrated can migrate back.

This is why parallel-live is specified with both contracts fully sealed during the transition rather than the old one being placed in a deprecated state. Deprecation would foreclose the return path. Maintaining both as sealed means the return path is always open, at the cost of maintaining two contracts in parallel until the transition is confirmed successful.

20.2 Decision Criteria for Rollback

Four conditions that collectively justify invoking rollback:

28. The new contract has been found to have a flaw that cannot be addressed by extension or small adjustment — it would require significant redesign
29. Consumers that have already migrated are experiencing issues that would not have occurred on the old contract
30. Continuing the transition would compound the problem rather than resolve it
31. Returning to the old contract while the flaw is addressed is operationally feasible

Not every discovered problem with a new contract warrants rollback. Many can be addressed in place — an extension to the new contract, a patch to the implementation, a documentation clarification. Rollback is the response when the contract itself is wrong, not when the implementation has a bug.

20.3 The Rollback Protocol

When rollback is invoked, the sequence is:

32. SRB convenes (emergency session if the issue is urgent) to evaluate the situation and formally approve rollback.
33. Executive notification — rollback is significant enough to require senior engineering leadership awareness, even when not requiring their approval.
34. New contract status changes to `rolled-back`. It is not retired — it remains in the registry with its full history, and may be revisited later after the flaw is understood. But it is no longer active for consumers.
35. Migrated consumers return to the old contract. The platform team assists with the reverse migration; the tooling that supported forward migration can usually support reverse migration with modest adaptation.

36. Old contract status returns to `sealed` (from `parallel-live-with-successor`). The parallel-live relationship is dissolved.
37. A formal post-mortem is conducted. What was missed in the original design review? What signals in early migration were under-weighted? The lessons feed into future replacement designs.

20.4 After Rollback

A rolled-back contract is not forever dead. After the flaw is understood and a corrected design is proposed, a new parallel-live transition can begin — with a new contract (v3, not v2 again), referencing the rolled-back v2 in its history as a lesson learned. The registry preserves the full story: v1 sealed, v2 proposed and rolled back, v3 sealed and successfully transitioning.

This transparency is a feature. Future engineers looking at the history of `auth.user\_session` will see the rollback. They will read the post-mortem. They will understand why v3 took the shape it did. The institutional memory persists, and the next replacement attempt inherits the learning.

*Rollback is not failure. Proceeding with a flawed replacement is failure.
Rollback is the discipline that prevents the flawed replacement from
becoming permanent.*

21. Major Version Bump — Detailed Protocol

The Major Version Bump is the rarest and most consequential evolution event. The whitepaper described it in principle. This section provides the operational protocol — the actual steps an organisation follows when a portion of its sealed architecture must be fundamentally re-specified because the purpose it served has changed.

An MVB is rare by design. An enterprise might undergo one every five to ten years, affecting a subset of its contracts. The heavy ceremony reflects the stakes: an MVB is the organisation acknowledging that what was previously durable is now contingent. Done badly, an MVB creates long-term architectural debt. Done well, it is how the system accommodates genuine transformation.

21.1 Triggers for an MVB

Four recognisable triggers typically warrant an MVB:

- Regulatory transformation — a new regulatory regime fundamentally changes what the system must do. Example: a major healthcare data regulation that redefines what 'patient consent' means, affecting every contract that touches patient data.
- Business model pivot — the enterprise shifts its revenue model, market, or value proposition in ways that make its current sealed contracts serve the wrong purpose. Example: transitioning from a licence-based software business to a usage-based one.
- Technology foundation shift — a platform technology that was embedded in sealed contract design becomes untenable. Example: shifting from a relational-database-centric architecture to a graph-database-centric one for a relationship-heavy domain.
- Accumulated fundamental flaws — extension after extension has revealed that multiple invariants in the sealed contracts were subtly wrong, and patching them all individually would be more disruptive than re-specifying from scratch.

A trigger that is not on this list is usually a sign that something less than MVB is warranted. 'We want a new feature' is extension. 'We want a cleaner design' is parallel-live replacement. MVB is reserved for transformations that genuinely cross the line where the old contracts cannot be brought forward in any form.

21.2 The Seven Stages

Stage 1 — Proposal and executive sponsorship

An MVB is proposed in writing. The proposal identifies the trigger, explains why lesser mechanisms are insufficient, and estimates the scope (which contracts are affected, what the new contracts will need to address, what the transition cost will be). An executive sponsor — CTO, VP Engineering, or equivalent — must co-sponsor the proposal. Without executive sponsorship, an MVB does not advance.

Stage 2 — SRB review and unanimous approval

The SRB reviews the proposal. MVB approval requires unanimity, not majority. If any SRB member dissents with a substantive concern, the proposal does not advance; the concern must be addressed or the proposal must be restructured. This is deliberate — MVB is too consequential for a divided Board to proceed.

Unanimity is not theatre. Dissent should be heard and understood. A dissenter may have a legitimate perspective that the majority has missed. The outcome should be that the Board either addresses the concern or declines to proceed; it is not that the dissenter is overridden.

Stage 3 — Review period

A formal review period follows approval. Minimum 60 days, typically 90. During this period, the current contracts remain fully sealed and untouched. No implementations begin work against new contracts. The purpose of the review period is to surface concerns that were not visible during initial review, to consult with stakeholders across the organisation, and to allow time for the magnitude of the change to be absorbed by affected teams.

Stage 4 — Replacement contract design

During the review period (or immediately after, depending on organisation practice), the replacement contracts are designed. This is substantial architectural work. Each contract being replaced typically produces one or more new contracts in the replacement set. Each new contract has a full intent document. The full set is reviewed as a coherent whole, because contracts in the replacement set often have dependencies on each other.

Stage 5 — Parallel-live transition

The new contracts are sealed. The old contracts remain sealed (in the `parallel-live-with-successor` state). Both sets are operational. Consumers migrate over time — typically 6–12 months for enterprise systems, longer for safety-critical or heavily regulated domains. The migration follows the parallel-live playbook (Section 19), with the additional complexity that multiple contracts are transitioning simultaneously.

Stage 6 — Retirement of the old contract set

Once migration is complete, the old contracts enter the retirement protocol (Section 22). They are preserved in the archive. Their status becomes `retired`. The parallel-live relationship is formally dissolved.

Stage 7 — Architectural version increment

The organisation's overall architectural version increments. This is a public-facing signal — internal documentation, partner communications, compliance filings all reflect that the architecture has undergone a major generational change. The increment should be accompanied by a public summary of what changed and why, preserved as part of the organisation's architectural history.

21.3 What to Watch For

Three patterns indicate an MVB is off-track and should be paused for reconsideration:

- The replacement contract set has become significantly larger than the original set, suggesting that simplicity was lost in the redesign
- Consumer migration stalls across multiple teams simultaneously, suggesting the new contracts do not serve real needs as well as hoped
- The transition window extends beyond 18 months without clear progress, suggesting the MVB may have been premature or the change was not as fundamental as believed

Each of these is a signal for the SRB to convene and review. Sometimes the correct response is to continue with adjustments. Sometimes the correct response is to pause the transition and reconsider whether the MVB was the right choice. Occasionally — rarely but possibly — the correct response is to abort the MVB entirely and return to the pre-MVB state, treating the exercise as a learning experience rather than a failed transformation.

22. Retirement Procedures

Every sealed contract eventually ends. Either its consumers migrate away, its purpose ceases to exist, or a Major Version Bump supersedes it. How contracts end matters almost as much as how they are sealed. A retirement performed carelessly leaves the organisation without the institutional knowledge the contract represented. A retirement performed with appropriate ceremony preserves that knowledge even as the contract itself leaves active service.

22.1 When Retirement Is Appropriate

Four conditions, any one of which justifies retirement:

38. Consumer count has been zero for at least one quarterly review cycle. The contract has no active users, and no new consumers are reasonably expected.
39. A parallel-live transition has completed. The contract has been superseded by a new version; migration is done; the old contract is no longer needed.
40. The domain the contract addressed no longer exists within the organisation. Example: a contract for a feature whose product line was divested.
41. An MVB has retired the contract as part of a broader architectural generation change.

Contracts that merely have low usage are not retirement candidates. A contract with two active consumers is still active — retirement would break those consumers. Low usage may trigger conversation about whether the contract is still needed, but retirement requires zero active consumers.

22.2 The Retirement Protocol

Retirement is a governed event, not a silent file deletion:

42. Confirm zero active consumers — the Contract Owner and the Platform Team together confirm that the contract has no active usage. Tooling examines registry references, live code paths, and integration partners to validate.
43. Retirement proposal — the Contract Owner drafts a retirement notice. The notice includes: reason for retirement, confirmation of zero consumers, date of proposed retirement, archive location where the contract will be preserved.
44. SRB witness — the next quarterly SRB meeting witnesses the retirement. The SRB verifies the proposal; formal dissent is rare at this stage but possible, typically if an SRB member is aware of a consumer the Owner did not know about.
45. Retirement execution — the contract's status is changed from `sealed` (or `parallel-live-with-successor`) to `retired`. The change is recorded in the contract's change history. The contract file remains in the registry.
46. Archive confirmation — the archive of retired contracts is updated. The contract's intent document, its full change history, its final state, and the retirement rationale are preserved. Future engineers and auditors can retrieve the full record.
47. Registry update — the central manifest is updated to reflect the retirement. Consumers of the manifest see that the contract is no longer active.

22.3 What Is Preserved, What Is Removed

After retirement, the contract retains a specific set of properties:

Element	Preserved	Notes
Contract YAML file	Yes	Retained in registry with status: retired
Intent document	Yes	Preserved unchanged; future readers understand the original thinking
Change history	Yes	The full log including the retirement event
Implementations referencing the contract	Removed over time	Following the migration completion; implementations are the transient layer
Verifier configurations for the contract	Removed	No active verification needed for retired contract
Dependencies from other contracts	Updated	Dependencies are rewritten to reference the successor (if parallel-live) or simply removed

The pattern: the durable artefacts (contract and intent) are preserved indefinitely. The transient artefacts (implementations, verifier configs) are removed. The contract is not erased from organisational memory; it is moved from active service to honoured retirement.

22.4 The Archive

The archive of retired contracts is itself an important artefact. It is the organisation's architectural memory — the record of what has existed and what it served. New architects joining the organisation can read the archive to understand not just the current state but the history that produced it.

Archive discipline:

- The archive is append-only. Once a contract is retired and archived, its record is not edited.
- The archive is searchable. Engineers can find retired contracts by seal_id or by domain, to understand past designs.
- The archive is included in the governance portfolio. Retired contracts are visible to the SRB and to future architectural reviews.
- The archive is preserved across organisational changes. If the organisation is acquired, divested, or restructured, the archive travels with the system it documents.

22.5 A Note on Dignity

The word 'dignity' might seem out of place in a technical handbook. But retirement ceremony serves a purpose that is difficult to articulate in purely technical language. Contracts that are quietly deleted when convenient encode an organisational attitude: that architectural artefacts are disposable, that past work does not matter, that institutional memory is not valued.

Contracts that are retired with ceremony — with explicit acknowledgement, preserved history, and archived intent — encode a different attitude: that the work of specifying and maintaining these artefacts mattered, that the people who created them contributed something worth preserving, and that the organisation values the continuity of its architectural record across generations of engineers.

The care with which a system ends its sealed artefacts reveals how seriously it took them in the first place.

This is not sentimental. It is architectural discipline. Organisations that retire contracts carelessly produce engineers who seal contracts carelessly, because the signal is clear: none of this really matters. Organisations that retire with appropriate ceremony produce engineers who seal with appropriate seriousness. The ceremony is how the culture of the discipline is transmitted across generations of practitioners.

Closing

This handbook has addressed four operational dimensions of SEA adoption: measurement, enforcement, verification, and evolution. It has done so with the specificity that distinguishes operational guidance from philosophical argument — concrete metrics with weights and thresholds, specific enforcement mechanisms with configuration details, compliance states with response criteria, evolution patterns with recognised anti-patterns.

The guidance is deliberately incomplete. Team-scale operations, SDLC integration patterns, reference templates, and worked sector examples were scoped out of this volume. They will appear in a second volume, after early adoption has revealed what this volume missed. The choice to defer rather than speculate is intentional — documentation that runs ahead of adoption experience produces confident-sounding guidance that becomes a hostage to fortune.

The operational guidance here will be refined by use. Metrics may need re-weighting. Enforcement patterns may need adaptation to specific toolchains. The five compliance states may need additional cases. The evolution mechanisms may need patterns not catalogued here. None of this is a failure of the handbook; it is the normal course of an operational document's maturation.

What should be stable is the underlying approach: measurement that tracks the durable layer, enforcement that is layered, verification that is richer than binary, evolution that is honest about what it is doing. Whatever the specifics become, the approach is durable even as the details change.

*A handbook without its underlying philosophy becomes mere procedure.
A philosophy without its operational handbook remains aspiration.
Together they become discipline.*

The SEA 1.0 whitepaper provided the philosophy. This handbook provides the first volume of operations. Together they form a starting point — not a completed system. What they become in the hands of the organisations that adopt them is the next chapter, one that this author cannot write alone.

— End of Volume 1 —

SEA Implementation Handbook · Volume 1
Measurement · Enforcement · Verification · Evolution
Suneesh Sidharthan · suneesh.sidharthan80@gmail.com · +91 9986898076
Released under CC BY 4.0 · April 2026